LangChain Coding Revision & Master Application Guide

GenAI Engineer Placement Handbook: Complete Architecture, Projects 1-8 & Practice

PHASE 1: Repository Learning Inventory & Gap Analysis

This inventory is grounded strictly in the verified code inside your repository (`d:\langchain`). No assumptions are made.

Module / Topic	Files Implemented in Your Repo	Status	Mastery & Knowledge Gaps
Chat Models & Providers	`LLMS/llm_demo.py` `ChatModels/chat_model_openai.py` `ChatModels/chat_model_gemini.py` `ChatModels/chat_model_antropic.py` `ChatModels/chat_model_hugging_face.py`	**Implemented**	Understands multi-provider configuration via OpenRouter and Gemini API. Gap: Handling provider fallbacks, streaming tokens, and Hugging Face serverless router migration.
Prompt Templates & UI	`Prompts/prompt_generator.py` `Prompts/static_prompt_ui.py` `Prompts/dynamic_prompt_ui.py` `Prompts/template.json`	**Implemented**	Understands variable binding, template validation, and JSON serialization with Streamlit. Gap: Few-shot prompting, partial variables, and ChatPromptTemplate pipeline syntax.
Messages & Chat History	`chatbot/messages.py` `chatbot/chat_prompt_template.py` `chatbot/chatbot.py` `Prompts/message_placeholder.py` `Prompts/chat_history.txt`	**Implemented**	Understands SystemMessage, HumanMessage, AIMessage, and terminal while-loop history. Gap: Safe serialization (avoiding ast.literal_eval), SQLite persistence, and context-window pruning.
Embeddings & Similarity	`EmbeddedModels/embedding_openai_query.py` `EmbeddedModels/embedding_openai_docx.py` `EmbeddedModels/embedding_hf_local.py` `EmbeddedModels/document_similarity.py`	**Implemented**	Understands OpenAI and local HuggingFace embeddings, vector dimensionalities, and Scikit-learn cosine similarity ranking. Gap: Vector databases (FAISS, Chroma), ANN indexing, and MMR.
Structured Output & Pydantic	*None currently in repository*	**Not Started**	High-priority gap. Industry projects demand JSON validation via `model.with_structured_output(Schema)`.
Document Loaders & Chunking	*None currently in repository*	**Not Started**	Prerequisite for production RAG. Must learn `PyPDFLoader` and `RecursiveCharacterTextSplitter`.
End-to-End RAG Chains	*None currently in repository*	**Not Started**	Core requirement for AI Engineer interviews. Combining vector retrievers with generation chains.
Tool Calling & Agents	*None currently in repository*	**Not Started**	Required for AI Assistants: defining `@tool`, binding to models, and handling tool execution loops.

PHASE 3: Project Roadmap (Projects 1 to 8)

Below is the complete architectural guide, reference code, execution workflow, testing, and interview prep for all 8 roadmap projects.

Project 1: Basic Multi-Provider LLM Gateway

1. Problem Statement: Build a unified LLM client that takes user prompts and queries multiple providers (OpenAI, Gemini, Anthropic) with streaming output and automatic fallback.

2. Required Components: `ChatOpenAI`, `ChatGoogleGenerativeAI`, `with_fallbacks`, `stream()`.

3. Folder Structure:

```
project1_gateway/
├── config.py            # API key loader
├── gateway.py           # Fallback model definition
└── main.py              # CLI streaming interface
```

4. Complete Reference Code:

```
# gateway.py
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_google_genai import ChatGoogleGenerativeAI
import sys

load_dotenv()

primary_model = ChatOpenAI(
    model="openai/gpt-4o-mini",
    base_url="https://openrouter.ai/api/v1",
    max_retries=1,
    timeout=5.0
)

fallback_model = ChatGoogleGenerativeAI(
    model="gemini-3.8-flash",
    max_retries=1
)

resilient_gateway = primary_model.with_fallbacks([fallback_model])

def stream_query(prompt: str):
    print("AI: ", end="", flush=True)
    for chunk in resilient_gateway.stream(prompt):
        print(chunk.content, end="", flush=True)
    print()

if __name__ == "__main__":
    stream_query("Why is sky blue in 2 sentences?")
```

5. Execution Workflow: The user passes a string. `stream()` attempts connection to OpenRouter. If successful, tokens stream to stdout. If OpenRouter returns 429/500 or times out after 5s, `resilient_gateway` catches the error and streams seamlessly from Gemini.

6. Interview Question: *"What is the memory and latency difference between invoke() and stream()?"*
Answer: `invoke()` blocks until the complete generation is finished, buffering the entire text in memory before returning an `AIMessage`. `stream()` uses HTTP chunked transfer encoding (SSE), yielding `AIMessageChunk` tokens as they arrive from the model, dramatically reducing Time-To-First-Token (TTFT) and improving perceived user experience.

Project 2: Dynamic Prompt Generator & Validator

1. Problem Statement: Build a parameterized code review assistant that takes language, code snippet, and seniority level, enforcing strict review formatting.

2. Required Components: `ChatPromptTemplate`, `SystemMessagePromptTemplate`, `HumanMessagePromptTemplate`, `ChatOpenAI`, `StrOutputParser`.

3. Architecture Diagram:

```
Input Dict {'lang': 'py', 'code': '...'} ---> [ChatPromptTemplate] ---> [ChatOpenAI] ---> [StrOutputParser]
---> Clean String
```

4. Complete Reference Code:

```
# code_reviewer.py
from dotenv import load_dotenv
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

load_dotenv()

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a Principal Software Engineer reviewing {language} code for a {seniority} engineer
    ("human", "Review this code snippet:\n\n{code_snippet}")
])

model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")
chain = prompt | model | StrOutputParser()

sample_code = """
def get_users():
    users = []
    for i in range(1000):
        users.append({'id': i, 'name': f'user_{i}'})
    return users
"""

review = chain.invoke({
    "language": "Python",
    "seniority": "Junior",
    "code_snippet": sample_code
})
print("Review Output:\n", review)
```

5. Key Component: `StrOutputParser()` automatically strips the `AIMessage` metadata container and extracts the plain string `.content`, allowing chains to output directly to users.

Project 3: Conversational Chatbot with Sliding-Window History

1. Problem Statement: Build a multi-turn chatbot with system persona guidance that automatically trims older conversation turns to stay within token limits.

2. Required Components: `ChatPromptTemplate`, `MessagesPlaceholder`, `HumanMessage`, `AIMessage`.

3. Complete Reference Code:

```
# sliding_chatbot.py
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

load_dotenv()
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an empathetic customer support assistant for Acme Bank. Be concise."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{user_input}")
])

model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")
chain = prompt | model

history = []
MAX_TURNS = 6  # Keep last 6 messages (3 user + 3 AI turns)

def chat(user_text: str) -> str:
    global history
    response = chain.invoke({
        "chat_history": history[-MAX_TURNS:],
        "user_input": user_text
    })
    history.append(HumanMessage(content=user_text))
    history.append(AIMessage(content=response.content))
    return response.content

if __name__ == "__main__":
    print("Bot ready! Type 'exit' to quit.")
    while True:
        txt = input("User: ")
        if txt.lower() == "exit":
            break
        print("AI:", chat(txt))
```

Project 4: Persistent Chatbot with SQLite & Session IDs

1. Problem Statement: Multi-user chatbot where each user has a unique `session_id` and conversation history persists across server reboots using SQLite.

2. Required Components: SQLite database, `messages_to_dict`, `messages_from_dict`, `ChatPromptTemplate`.

3. Reference Code:

```
# persistent_chat.py
import sqlite3
import json
from dotenv import load_dotenv
from langchain_core.messages import messages_to_dict, messages_from_dict, HumanMessage, AIMessage
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_openai import ChatOpenAI

load_dotenv()

class SQLiteSessionHistory:
    def __init__(self, db_file="sessions.db"):
        self.conn = sqlite3.connect(db_file, check_same_thread=False)
        self.conn.execute("CREATE TABLE IF NOT EXISTS history (session_id TEXT PRIMARY KEY, msgs TEXT)")
        self.conn.commit()

    def get(self, session_id: str):
        cur = self.conn.cursor()
        cur.execute("SELECT msgs FROM history WHERE session_id = ?", (session_id,))
        row = cur.fetchone()
        return messages_from_dict(json.loads(row[0])) if row else []

    def save(self, session_id: str, messages: list):
        cur = self.conn.cursor()
        raw = json.dumps(messages_to_dict(messages))
```

```
        cur.execute("INSERT INTO history (session_id, msgs) VALUES (?, ?) ON CONFLICT(session_id) DO UPDATE
        self.conn.commit()

db = SQLiteSessionHistory()
model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an AI tutor."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}")
])
chain = prompt | model

def handle_user_query(session_id: str, query: str) -> str:
    past = db.get(session_id)
    resp = chain.invoke({"history": past[-10:], "input": query})
    past.extend([HumanMessage(content=query), AIMessage(content=resp.content)])
    db.save(session_id, past)
    return resp.content
```

Project 5: Structured Output Application (Pydantic Schema Validation)

1. Problem Statement: Extract entity information (person name, email, sentiment, action items) from unstructured customer feedback emails into a validated Python object.

2. Required Components: `pydantic.BaseModel`, `Field`, `model.with_structured_output()`.

3. Complete Reference Code:

```
# structured_extractor.py
from pydantic import BaseModel, Field
from typing import List, Optional
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

class FeedbackAnalysis(BaseModel):
    customer_name: Optional[str] = Field(description="Name of the customer if mentioned")
    sentiment: str = Field(description="Must be POSITIVE, NEUTRAL, or NEGATIVE")
    urgency_rating: int = Field(description="1 to 5 scale where 5 is critical")
    issues_identified: List[str] = Field(description="Key complaints or features requested")
    action_item: str = Field(description="Recommended next step for support team")

model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")
structured_llm = model.with_structured_output(FeedbackAnalysis)

email_text = """
Hi Support,
I am Rajesh Kumar. I've been trying to export my invoices to PDF since yesterday,
but the system throws a 500 error every time! This is blocking our tax filing today.
Please fix this immediately!
"""

result: FeedbackAnalysis = structured_llm.invoke(f"Analyze this customer message:\n{email_text}")

print(f"Customer: {result.customer_name}")
print(f"Sentiment: {result.sentiment}")
print(f"Urgency: {result.urgency_rating}/5")
print(f"Issues: {result.issues_identified}")
print(f"Action: {result.action_item}")
```

4. Key Engineering Advantage: `with_structured_output()` leverages OpenAI's function calling / JSON mode under the hood, guaranteeing 100% schema compliance without manual regex parsing.

Project 6: Document Question-Answering (Loaders & Splitters)

1. Problem Statement: Ingest a large text/markdown document, split it into semantically meaningful chunks with overlap, vectorize them, and answer user questions directly from the text.

2. Required Components: `RecursiveCharacterTextSplitter`, `HuggingFaceEmbeddings`, `FAISS`, `ChatOpenAI`.

3. Complete Reference Code:

```
# doc_qa.py
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

policy_text = """
ACME Vacation Policy 2026:
Employees are entitled to 20 days paid leave per calendar year.
Leave must be requested at least 14 days in advance via the HR Portal.
A maximum of 5 unused leave days can be carried over into the next fiscal year.
Emergency medical leave requires a registered physician note within 48 hours.
"""

# 1. Chunking
splitter = RecursiveCharacterTextSplitter(chunk_size=150, chunk_overlap=30)
chunks = splitter.create_documents([policy_text])

# 2. Embedding & Vectorstore
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

# 3. QA Prompt
prompt = ChatPromptTemplate.from_template("""
Answer the question strictly based on the provided context below.
Context:
{context}

Question: {question}
Answer:
""")

model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")

query = "How many days can I carry over to next year?"
retrieved_docs = retriever.invoke(query)
context_str = "\n\n".join([d.page_content for d in retrieved_docs])

chain = prompt | model | StrOutputParser()
answer = chain.invoke({"context": context_str, "question": query})
print("Answer:", answer)
```

Project 7: Production RAG Application with Source Citations

1. Problem Statement: Build a complete enterprise RAG pipeline using modern LCEL syntax that retrieves relevant documents, generates grounded answers, and returns exact source citations.

2. Required Components: `RunnableParallel`, `RunnablePassthrough`, `as_retriever()`, `ChatPromptTemplate`, `ChatOpenAI`.

3. Architecture Diagram:

```
+---> [Retriever] ---> [Format Docs] ---> {context} ---+ [User Question] -------| |---> [Prompt] ---> [LLM] ---> Output +---------------------------------------> {question} --+
```

4. Complete Reference Code:

```
# enterprise_rag.py
from dotenv import load_dotenv
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

load_dotenv()

# Sample Knowledge Base
raw_data = [
    Document(page_content="Kubernetes pods communicate via an overlay network where each pod receives a uni
    Document(page_content="ConfigMaps bind non-confidential configuration data directly to Pod container en
    Document(page_content="Horizontal Pod Autoscaler (HPA) scales replica sets based on observed CPU utili
]

embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
vectorstore = FAISS.from_documents(raw_data, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

def format_docs(docs):
    return "\n\n".join([f"[Source: {d.metadata['source']} p.{d.metadata['page']}]: {d.page_content}" for d

rag_prompt = ChatPromptTemplate.from_template("""
You are a Cloud Infrastructure Architect. Answer the question using ONLY the provided context.
Include source citations in your answer.

Context:
{context}

Question: {question}
Answer:
""")

model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1", temperature=0)

# Modern LCEL RAG Chain:
rag_chain = (
    RunnableParallel({
        "context": retriever | format_docs,
        "question": RunnablePassthrough()
    })
    | rag_prompt
    | model
    | StrOutputParser()
)

print(rag_chain.invoke("How does Kubernetes scale pods automatically?"))
```

Project 8: Tool-Using AI Assistant (Function Calling & Agentic Loop)

1. Problem Statement: Build an autonomous AI assistant equipped with real-world calculation and weather tools that inspects user queries, determines when external tools are needed, calls the tools, and synthesizes final answers.

2. Required Components: `@tool`, `bind_tools()`, `ToolMessage`, `AIMessage`.

5. Complete Reference Code:

```
# tool_assistant.py
from dotenv import load_dotenv
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, ToolMessage
from langchain_openai import ChatOpenAI

load_dotenv()

# 1. Define custom tools with docstrings (crucial for model comprehension)
@tool
def calculate_emi(principal: float, rate_percent: float, tenure_months: int) -> float:
    """Calculates monthly loan EMI given principal amount, annual interest rate percentage, and tenure in m
    r = (rate_percent / 12) / 100
    emi = (principal * r * ((1 + r) ** tenure_months)) / (((1 + r) ** tenure_months) - 1)
    return round(emi, 2)

@tool
def get_stock_price(ticker: str) -> float:
    """Retrieves the current stock price for a company ticker symbol (e.g. AAPL, MSFT, INFY)."""
    dummy_prices = {"AAPL": 225.50, "INFY": 21.30, "GOOGL": 180.20}
    return dummy_prices.get(ticker.upper(), 100.0)

tools = [calculate_emi, get_stock_price]
tool_map = {t.name: t for t in tools}

# 2. Bind tools to the model
model = ChatOpenAI(model="openai/gpt-4o-mini", base_url="https://openrouter.ai/api/v1")
model_with_tools = model.bind_tools(tools)

# 3. Agent Execution Engine
def run_agent(query: str):
    messages = [HumanMessage(content=query)]
    response = model_with_tools.invoke(messages)
    messages.append(response)

    # Check if model requested tool execution:
    if response.tool_calls:
        for tool_call in response.tool_calls:
            tool_name = tool_call["name"]
            tool_args = tool_call["args"]
            tool_call_id = tool_call["id"]

            print(f"[Agent calling {tool_name} with {tool_args}]")
            tool_fn = tool_map[tool_name]
            output = tool_fn.invoke(tool_args)

            # Append ToolMessage result
            messages.append(ToolMessage(content=str(output), tool_call_id=tool_call_id))

        # Final synthesis call
        final_answer = model_with_tools.invoke(messages)
        return final_answer.content
    return response.content

print(run_agent("What is the monthly EMI for a car loan of $30,000 at 7.5% annual interest over 48 months?")
```

PHASE 4: Coding & Interview Practice Challenges

Test your knowledge with these 5 high-frequency technical interview coding challenges.

Challenge 1: Safe Message History Parsing

Problem: Replace custom text file parsing (`ast.literal_eval`) with a production JSON serialization method that saves and restores a list containing `SystemMessage`, `HumanMessage`, and `AIMessage` without losing attributes.

Solution:

```
from langchain_core.messages import messages_to_dict, messages_from_dict
import json

def save_chat(filepath: str, msgs: list):
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump(messages_to_dict(msgs), f)

def load_chat(filepath: str) -> list:
    with open(filepath, "r", encoding="utf-8") as f:
        return messages_from_dict(json.load(f))
```

Challenge 2: Dynamic Few-Shot Prompt Selector

Problem: How do you dynamically select the top 2 most semantically relevant examples for a prompt from a pool of 50 examples instead of hardcoding them?

Solution: Use `SemanticSimilarityExampleSelector` from `langchain_core.example_selectors` backed by a FAISS vector store.

Challenge 3: Hybrid Search RAG

Problem: In an interview, explain how you combine keyword BM25 search with Dense vector embeddings to prevent hallucination on rare product SKUs.

Solution: Use LangChain's `EnsembleRetriever` combining `BM25Retriever` and `FAISS.as_retriever()` with weighted reciprocal rank fusion (e.g. `weights=[0.5, 0.5]`).

PHASE 5: LangChain 1.x Version Compatibility & Modern Best Practices

Your environment runs modern LangChain 1.x packages (`langchain 1.3.17`, `langchain-core 1.6.0`). Here are the essential migration rules:

Outdated / Legacy 0.x Pattern	Modern 1.x Equivalent	Why It Was Changed
`from langchain.chat_models import ChatOpenAI`	`from langchain_openai import ChatOpenAI`	Partner packages are decoupled to isolate dependencies and speed up imports.
`LLMChain(llm=model, prompt=prompt)`	`chain = prompt | model`	LCEL (LangChain Expression Language) standardizes all components on the Runnable protocol.
`RetrievalQA.from_chain_type(...)`	`RunnableParallel({"context": retriever, "question": ...}) | prompt | model`	Old monolithic chains hid prompts and context formatting, preventing inspection and debugging.
`initialize_agent(..., AgentType.ZERO_SHOT_REACT)`	`model.bind_tools(tools)` or LangGraph	Modern LLMs have native function calling; brittle prompt-based regex ReAct parsing is deprecated.

PHASE 6: Final Master Learning Dashboard

Concept / Area	Mastery Status	Repository Evidence	Recommended Action
Chat Models & Multi-Provider API	**Implemented**	`ChatModels/` (OpenAI, Gemini, Claude via OpenRouter)	Implement streaming in CLI; handle provider fallbacks.
Prompt Templates & Serialization	**Implemented**	`Prompts/` (PromptTemplate, dumps, load_prompt)	Build FewShotPromptTemplate exercise.
Messages & In-Memory Chatbot	**Implemented**	`chatbot/` (messages.py, chatbot.py)	Upgrade to SQLite session persistence.
Dense Embeddings & Vector Similarity	**Implemented**	`EmbeddedModels/` (OpenAI, HF local, Cosine Sim)	Integrate FAISS vector store with Document loaders.
Structured Output & Pydantic	**Practised**	Master Guide Project 5 reference solution	Implement customer feedback parser script from scratch.
Document Chunking & Loaders	**Practised**	Master Guide Project 6 reference solution	Test PyPDFLoader on local PDF files.
End-to-End RAG Pipeline	**Practised**	Master Guide Project 7 reference solution	Build full RAG CLI with source citations.
Function Calling & Tool Agents	**Practised**	Master Guide Project 8 reference solution	Build a multi-tool agent using `@tool` and `bind_tools`.